BMH309 Mobil Cihaz Programlama

Dart ile nesne yönelimli programlama: sınıf, kurucu, kapsülleme, kalıtım, polimorfizm

Dr. Öğr. Üyesi Fesih Keskin

Bilgisayar Mühendisliği

Ders formatı: Slayt → VS Code → Dönüş ve tartışma

Bu haftanın öğrenme hedefleri

1. Sınıf ve nesne kavramlarını açıklamak; alan ve metot içeren sınıflar yazmak.
2. Kurucular yazmak; `this` anahtar kelimesinin anlamını açıklamak.
3. İsimli kurucular, başlatıcı listeleri ve `factory` kurucular kullanmak.
4. `_` ile private alanlar tanımlamak; getter ve setter ile kontrollü erişim sağlamak.
5. `extends` ile kalıtım kurmak; `super` ile üst sınıfı çağırmak.
6. `@override` ile metotları yeniden tanımlamak.
7. Polimorfizmi açıklamak; `is`, `is!` ve `as` ile tip kontrolü yapmak.

Dersi nasıl işleyeceğiz?

1. Slayt	2. VS Code	3. Dönüş
Kavram	**Uygulama**	**Tartışma**
Kural, şekil veya tahmin sorusu	Dosyayı yaz, çalıştır, değiştir	Çıktı, hata ve alternatif çözüm

Sunumda	**Ders notunda**
Başlıktaki numara: **5.3** | **Getter**	Bölüm **5.3**
VS Code | **5.3–5.5** slaytındaki dosyalar	Aynı adlı kod blokları (`05_06_getter.dart` gibi)
Dönüş | **5.3–5.5** slaytındaki çıktılar	Kodun hemen altındaki **Çıktı** bloğu
Alt bilgi çubuğu	O an işlenen bölüm

Yol haritası ve önerilen zaman planı

Blok	Ders notu bölümleri
Blok 1 (50 dk)	**1** Ödev 2 çözümleri (15 dk) · **2** Sınıf ve nesne · **3** Kurucular ve this
Blok 2 (50 dk)	**4** İsimli kurucular ve factory · **5** Kapsülleme · **6** Kalıtım
Blok 3 (50 dk)	**7** Override · **8** Polimorfizm ve is · **9** Cep Market v2 · **10** Ödeme problemi
Evde	**11** Özet · **12** Kendinizi test edin · **13** Ödev 3 · **14** Kaynaklar

Zaman daralırsa

Hızlı geçilebilecekler: **4.4** static, **4.5** önbellek örneği, **8.5** tip desenleri, **10**'un 3. çözümü. Hepsi ders notunda eksiksiz anlatılıyor.

Çalışma ilkeleri | AI açıklar, siz yazarsınız

- AI'ı kavram sormak, hata mesajını açıklatmak ve kodunuzu okutmak için kullanabilirsiniz.
- AI'a **kod yazdırmak yasaktır**. Claude Code plan modunda.
- Sınıf içi mini görevler **AI kapalı**.
- Önce çıktıyı tahmin edin, sonra çalıştırın.

Bu haftanın klasörü

`bmh309/hafta4/`

Bölüm 5.2'deki iki dosyalı örnek dışında her dosya tek başına çalışır.

Neden nesne yönelimli programlama?

Hafta 3: Veri dağınık

Fiyat bir Map'te, adet başka bir Map'te, ad anahtarlarda. Yeni bir bilgi (stok, kategori) için yeni bir Map ve fonksiyonlara bir parametre daha:

```
sepeteEkle(sepet, katalog, urun)
```

Hafta 4: Veri ve davranış bir arada

Bir ürünün tüm bilgileri ve onunla yapılabilecek işler tek bir yapıda:

```
sepet.ekle(sut, adet: 2)
sut.kargoUcreti()
```

Flutter'da her şey bir nesnedir

Ekrandaki her yazı, buton ve resim bir sınıftan üretilmiş nesnedir. Kendi ekranlarınızı `StatelessWidget`'tan **kalıtım** alarak yazacak ve `build` metodunu **override** edeceksiniz.

1 | Ödev 2 Çözümleri

İlk 15 dakika: fonksiyonlar ve koleksiyonlar

Ders notu: Bölüm 1 · Dosyalar: `01_s1` – `01_s5`

1 | Ödev 2: beş soru, beş ana fikir

Soru	Ana fikir
1 Alışveriş listesi	Erken çıkış ile doğrulama; `List.of` ile kopyalayıp sıralama (referans tuzağı)
2 Haftalık adım raporu	Sabit uzunluklu liste; varsayılan değerli isimli parametre `{int hedef = 8000}`
3 Ortak ilgi alanları	`intersection`, `union`, `difference`; `padRight` ile hizalama
4 Yorum analizi	Sayma kalıbı `m[k] = (m[k] ?? 0) + 1`; `continue`
5 Telefon rehberi	Her iş ayrı fonksiyonda; `remove` dönüş değeri; etiketli `break`

Bu haftaya köprü

Her soruda veri (liste, Map) ile fonksiyonlar **ayrı** duruyor: `urunEkle(sepet, ...)`, `kisiEkle(rehber, ...)`. Sınıflar bunları birleştirecek: `sepet.ekle(...)`, `rehber.ekle(...)`.

Dönüş | 1 · Soru 1 ve Soru 2

01_s1_alisveris_listesi.dart

```
Sepet boş.
Süt eklendi.
Ekmek eklendi.
Süt zaten sepette.
Ürün adı boş olamaz.
Armut eklendi.
Çay sepette bulunamadı.
Ekmek çıkarıldı.
Domates eklendi.
1. Süt
2. Armut
3. Domates
Alfabetik: [Armut, Domates, Süt]
Orijinal : [Süt, Armut, Domates]
```

01_s2_adim_raporu.dart

```
Toplam adım  : 59150
Ortalama     : 8450.0
En aktif gün : Cmt (12100 adım)
Hedef 8000   : 4 gün
Hedef 10000  : 2 gün
Pzt | ###### 6500
Sal | ######## 8200
Çar | ####### 7400
Per | ########## 10350
Cum | ##### 5600
Cmt | ############ 12100
Paz | ######### 9000
```

Dönüş | 1 · Soru 1 ve Soru 2

01_s1_alisveris_listesi.dart

```
Sepet boş.
Süt eklendi.
Ekmek eklendi.
Süt zaten sepette.
Ürün adı boş olamaz.
Armut eklendi.
Çay sepette bulunamadı.
Ekmek çıkarıldı.
Domates eklendi.
1. Süt
2. Armut
3. Domates
Alfabetik: [Armut, Domates, Süt]
Orijinal : [Süt, Armut, Domates]
```

01_s2_adim_raporu.dart

```
Toplam adım  : 59150
Ortalama     : 8450.0
En aktif gün : Cmt (12100 adım)
Hedef 8000   : 4 gün
Hedef 10000  : 2 gün
Pzt | ###### 6500
Sal | ######## 8200
Çar | ####### 7400
Per | ########## 10350
Cum | ##### 5600
Cmt | ############ 12100
Paz | ######### 9000
```

Tartışma

S1: `alfabetik` içinde `sepet.sort()` yazılsaydı son satır da sıralı çıkardı.
S2: `adimlar.add(7000)` → `Unsupported operation`: sabit uzunluklu liste.

Dönüş | 1 · Soru 3 ve Soru 4

01_s3_ilgi_alanlari.dart

```
Ortak ilgi alanları : {müzik, yazılım}
Tüm ilgi alanları   : {müzik, sinema, yazılım, fotoğraf, koşu, oyun, basketbol}
Yalnızca Ayşe       : {sinema, fotoğraf, koşu}
Yalnızca Mert       : {oyun, basketbol}
Benzerlik           : %28.57
Temizlenmiş etiketler: {yazılım, müzik, oyun}
```

01_s4_yorum_analizi.dart

```
Farklı kelime sayısı : 12
Toplam kelime sayısı : 16
En sık geçen         : hızlı (3 kez)
En az 2 kez geçenler : [uygulama, hızlı, güzel]
```

01_s3_ilgi_alanlari.dart

```
Ortak ilgi alanları : {müzik, yazılım}
Tüm ilgi alanları   : {müzik, sinema, yazılım, fotoğraf, koşu, oyun, basketbol}
Yalnızca Ayşe       : {sinema, fotoğraf, koşu}
Yalnızca Mert       : {oyun, basketbol}
Benzerlik           : %28.57
Temizlenmiş etiketler: {yazılım, müzik, oyun}
```

01_s4_yorum_analizi.dart

```
Farklı kelime sayısı : 12
Toplam kelime sayısı : 16
En sık geçen         : hızlı (3 kez)
En az 2 kez geçenler : [uygulama, hızlı, güzel]
```

Tartışma

S3: Benzerlik = 2 ortak / 7 toplam = %28.57 (Jaccard benzerliği). `Set` tekrarları kendiliğinden ayıklar.
S4: Metinde büyük `I` olsaydı `toLowerCase()` onu `ı` değil `i` yapardı.

Dönüş | 1 · Soru 5: Telefon rehberi

```
--- REHBER (2 kişi) ---
Seçiminiz: 1
Ad: Zeynep
Telefon: 0505 123 45 67
"Zeynep" eklendi.

--- REHBER (3 kişi) ---
Seçiminiz: 1
Ad: Can
Telefon: 12ab
Geçersiz telefon numarası.

--- REHBER (3 kişi) ---
Seçiminiz: 2
Aranacak ad: ay
Ayşe: 0532 111 22 33
```

```
--- REHBER (3 kişi) ---
Seçiminiz: 4
1. Ayşe: 0532 111 22 33
2. Mert: 0544 999 88 77
3. Zeynep: 0505 123 45 67

--- REHBER (3 kişi) ---
Seçiminiz: 3
Silinecek ad: Mert
"Mert" silindi.

--- REHBER (2 kişi) ---
Seçiminiz: 0
Rehberde 2 kişi var. Güle güle!
```

Dönüş | 1 · Soru 5: Telefon rehberi

```
--- REHBER (2 kişi) ---
Seçiminiz: 1
Ad: Zeynep
Telefon: 0505 123 45 67
"Zeynep" eklendi.

--- REHBER (3 kişi) ---
Seçiminiz: 1
Ad: Can
Telefon: 12ab
Geçersiz telefon numarası.

--- REHBER (3 kişi) ---
Seçiminiz: 2
Aranacak ad: ay
Ayşe: 0532 111 22 33
```

```
--- REHBER (3 kişi) ---
Seçiminiz: 4
1. Ayşe: 0532 111 22 33
2. Mert: 0544 999 88 77
3. Zeynep: 0505 123 45 67

--- REHBER (3 kişi) ---
Seçiminiz: 3
Silinecek ad: Mert
"Mert" silindi.

--- REHBER (2 kişi) ---
Seçiminiz: 0
Rehberde 2 kişi var. Güle güle!
```

Bu haftaya köprü

`rehber` Map'i her fonksiyona tek tek gönderiliyor. Bölüm 2'den sonra bunu bir `Rehber` sınıfına dönüştürmeyi deneyin.

2 | Sınıf ve Nesne

Veriyi ve davranışı tek bir yapıda toplamak

Ders notu: Bölüm 2 · Dosyalar: `02_01` – `02_07`

2.1 | Sorun: dağınık veri

Paralel listeler (02_01_dagik_veri.dart)

```dart
List<String> adlar = ['Süt', 'Ekmek', 'Peynir'];
List<double> fiyatlar = [32.50, 15.00, 145.00];
List<int> stoklar = [12, 0, 5];

adlar.add('Zeytin');
fiyatlar.add(120.00);
// stoklar.add(...) unutuldu!
```

Döngü 4. ürüne gelince ne olur?

2.1 | Sorun: dağınık veri

Paralel listeler (02_01_dagik_veri.dart)

```dart
List<String> adlar = ['Süt', 'Ekmek', 'Peynir'];
List<double> fiyatlar = [32.50, 15.00, 145.00];
List<int> stoklar = [12, 0, 5];

adlar.add('Zeytin');
fiyatlar.add(120.00);
// stoklar.add(...) unutuldu!
```

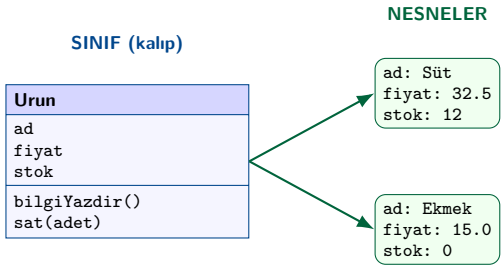
Döngü 4. ürüne gelince ne olur?

Sonuç

Üç ürün yazılır, sonra `RangeError`: `stoklar[3]` yok.

Bir ürünün bilgileri birbirinden kopuk üç yerde. İhtiyacımız: bir ürünün **tüm bilgilerini bir arada** tutan bir yapı.

2.2 | Sınıf bir kalıptır, nesne o kalıptan üretilir



Sınıf	Kalıp, tanım
Nesne	Somut örnek
Alan	Nesnenin verisi
Metot	Nesnenin yaptığı iş
.	Erişim: `sut.fiyat`

Benzetme

Sınıf bir evin **mimari planı**, nesneler o plana göre yapılmış **evlerdir**. Tek plandan istediğiniz kadar ev yapılır; her evin kapı rengi (verisi) farklı olabilir.

2.3–2.4 | İlk sınıf: alanlar ve metotlar

```
class Urun {
  String ad = '';
  double fiyat = 0;
  int stok = 0;

  bool sat(int adet) {
    if (adet > stok) return false;
    stok -= adet;
    return true;
  }
}

Urun peynir = Urun();
peynir.ad = 'Peynir';
peynir.sat(2);
```

- `class Urun` yeni bir **tip** tanımlar.
- `Urun()` yeni bir **nesne** oluşturur; `new` yazılmaz.
- Metot, **çağrıldığı nesnenin** alanlarını kullanır: `peynir.sat(2)`
- Alanların o anki değerleri = nesnenin **durumu**.
- Sınıf adı `UpperCamelCase`, alan ve metot `lowerCamelCase`.

VS Code | 2.1–2.4 · İlk sınıflar

Dosyalar

`02_01_dagik_veri.dart` — **Önce tahmin**

`02_02_ilk_sinif.dart` · `02_03_metotlar.dart` — **Birlikte yazalım**

1. 02_01'i çalıştırın; hatayı okuyun ve nedenini söyleyin.
2. `Urun` sınıfını yazın; iki nesne oluşturup `bilgiYazdir()` çağırın.
3. `stoktaVarMi`, `kdvliFiyat`, `stokEkle`, `sat` metotlarını ekleyin.
4. `peynir.sat(5)` neden reddedildi? Kural nerede yazılı?

Sorgu

Hafta 3'te `urunEkle(sepet, urun)` yazıyorduk. Şimdi `peynir.sat(2)`. Fark ne?

Dönüş | 2.1–2.4 · Çıktılar

02_01 (bir bölümü, ardından `RangeError`)

```
Süt: 32.5 TL, stok: 12
Ekmek: 15.0 TL, stok: 0
Peynir: 145.0 TL, stok: 5
```

02_02

```
Süt: 32.50 TL (stok: 12)
Ekmek: 15.00 TL (stok: 0)
```

02_03

```
Stokta var mı? true
KDV dahil: 174.00 TL
Peynir: 2 adet satıldı. Kalan: 1
Peynir: Yetersiz stok (1 adet var).
Peynir stoğuna 10 eklendi. Yeni stok: 11
Peynir: 5 adet satıldı. Kalan: 6
```

Dönüş | 2.1–2.4 · Çıktılar

02_01 (bir bölümü, ardından `RangeError`)

```
Süt: 32.5 TL, stok: 12
Ekmek: 15.0 TL, stok: 0
Peynir: 145.0 TL, stok: 5
```

02_02

```
Süt: 32.50 TL (stok: 12)
Ekmek: 15.00 TL (stok: 0)
```

02_03

```
Stokta var mı? true
KDV dahil: 174.00 TL
Peynir: 2 adet satıldı. Kalan: 1
Peynir: Yetersiz stok (1 adet var).
Peynir stoğuna 10 eklendi. Yeni stok: 11
Peynir: 5 adet satıldı. Kalan: 6
```

Tartışma

Veri ve onunla çalışan kod **aynı yerde**. Satış kuralı sınıfın içinde olduğu için programın neresinde satış yapılırsa yapılsın **aynı kural** uygulanır.

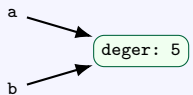
2.5–2.7 | toString, referans ve null safety

toString (2.5)

`print(u)` →
`Instance of 'Urun'`

Kendi `toString`'inizi yazın; hata ayıklarken tek `print` yeter.

Referans (2.6)



`b = a` kopya değil; aynı nesne.

Null safety (2.7)

`String ad;` başlangıç değeri yoksa derleme hatası.

Çözüm: varsayılan değer, `String?` ya da **kurucu**.

2.8 Nesne listeleri: `List<Urun> katalog` → paralel liste sorunu biter; ama her ürün 4 satırda oluşuyor. Çözüm: Bölüm 3.

VS Code | 2.5–2.8 · toString, referans, nesne listesi

Dosyalar

`02_04_tostring.dart` · `02_05_nesne_referans.dart`	**Birlikte yazalım**
`02_06_tuzak_alan_baslatma.dart`	**Hata mesajını oku**
`02_07_nesne_listesi.dart`	**Öğrenci yazsın**

1. `Urun` ve `Konum` nesnelerini yazdırın; farkı açıklayın.
2. `a == c` neden `false`? İçerikleri aynı değil mi?
3. 02_06'daki hatayı üç farklı yolla düzeltin.
4. `List<Urun>` oluşturup tabloyu `padRight`/`padLeft` ile hizalayın.

Dönüş | 2.5–2.8 · Çıktılar ve hata

02_04

```
Instance of 'Urun'
Konum(37.93, 41.94)
Teslimat noktası: Konum(37.93, 41.94)
```

02_05

```
a: 5, b: 5
a: 6
false
true
```

02_06 derleme hatası

```
Non-nullable instance field 'ad' must be
     initialized.
```

02_07

```
Süt         32.50 TL  12 adet
Ekmek       15.00 TL  TÜKENDİ
```

Dönüş | 2.5–2.8 · Çıktılar ve hata

02_04

```
Instance of 'Urun'
Konum(37.93, 41.94)
Teslimat noktası: Konum(37.93, 41.94)
```

02_05

```
a: 5, b: 5
a: 6
false
true
```

02_06 derleme hatası

```
Non-nullable instance field 'ad' must be
     initialized.
```

02_07

```
Süt         32.50 TL  12 adet
Ekmek       15.00 TL  TÜKENDİ
```

Hafta 3 ile aynı kural

Listeler gibi nesneler de **referansla** tutulur. Varsayılan ==, "aynı nesne mi?" diye bakar.

3 | Kurucular ve this

Nesneyi tek satırda ve eksiksiz oluşturmak

Ders notu: Bölüm 3 · Dosyalar: `03_01` – `03_08`

3.1–3.2 | Kurucu: gövdede atama neden çalışmaz?

Java / C# alışkanlığı (03_01)

```
class Urun {
  String ad;
  double fiyat;
  Urun(String ad, double fiyat) {
    this.ad = ad;       // çok geç!
    this.fiyat = fiyat;
  }
}
```

Dart'ta doğrusu (03_02)

```
class Urun {
  String ad;
  double fiyat;
  int stok;

  Urun(this.ad, this.fiyat, this.stok);
}

Urun sut = Urun('Süt', 32.5, 12);
```

Kural: Kurucunun gövdesi çalışmadan **önce** tüm null olamayan alanların değeri hazır olmalıdır. `this.ad` parametresi atamayı gövdeden önce yapar.

3.3 | this: şu anki nesne

```
class Kullanici {
  String ad;
  int puan = 0;

  Kullanici(this.ad);

  void puanEkle(int puan) {
    this.puan += puan;
  }
}
```

- `ayse.puanEkle(10)` çağrısında metodun içindeki `this`, Ayşe nesnesidir.
- `puan`: parametre, `this.puan`: alan.
- Ad çakışması yoksa `this` yazılmaz.

Tuzak (03_04): `ad = ad;`

İki `ad` de parametredir; parametre kendine atanır, alan değişmez. Program çalışır ama yanlış sonuç verir. Doğrusu: `this.ad = ad;`

VS Code | 3.1–3.3 · İlk kurucular

Dosyalar

`03_01_tuzak_kurucu_govde.dart`	**Hata mesajını oku**
`03_02_kurucu_kisa.dart` · `03_03_this.dart`	**Birlikte yazalım**
`03_04_tuzak_this.dart`	**Önce tahmin**

1. 03_01'deki hatayı okuyun; `this.ad` parametreleriyle düzeltin.
2. `Urun('Ekmek', 15, 0)`: 15 bir `int`; neden kabul edildi?
3. 03_04'ü çalıştırmadan önce çıktıyı tahmin edin.

Dönüş | 3.1–3.3 · Çıktılar ve hatalar

03_01 derleme hatası

```
Non-nullable instance field 'ad' must be
    initialized.
```

03_02

```
Süt: 32.50 TL (stok: 12)
Ekmek: 15.00 TL (stok: 0)
```

03_03

```
Ayşe: +10 puan (toplam: 10)
Mert: +5 puan (toplam: 5)
Ayşe: +25 puan (toplam: 35)
```

03_04 *tahmininiz?*

Dönüş | 3.1–3.3 · Çıktılar ve hatalar

03_01 derleme hatası

```
Non-nullable instance field 'ad' must be
     initialized.
```

03_02

```
Süt: 32.50 TL (stok: 12)
Ekmek: 15.00 TL (stok: 0)
```

03_03

```
Ayşe: +10 puan (toplam: 10)
Mert: +5 puan (toplam: 5)
Ayşe: +25 puan (toplam: 35)
```

03_04 *tahmininiz?*

```
Misafir
```

Dönüş | 3.1–3.3 · Çıktılar ve hatalar

03_01 derleme hatası

```
Non-nullable instance field 'ad' must be
    initialized.
```

03_02

```
Süt: 32.50 TL (stok: 12)
Ekmek: 15.00 TL (stok: 0)
```

03_03

```
Ayşe: +10 puan (toplam: 10)
Mert: +5 puan (toplam: 5)
Ayşe: +25 puan (toplam: 35)
```

03_04 *tahmininiz?*

```
Misafir
```

Neden kısa kurucu daha iyi?

Tek satırda nesne oluşur ve bir alanı unutmak **imkânsızdır**: Argüman eksikse program derlenmez.

3.4–3.5 | İsimli parametreler ve başlatıcı listesi

İsimli parametreler (3.4)

```
Urun({
  required this.ad,
  required this.fiyat,
  this.stok = 0,
  this.kategori = 'Genel',
});

Urun(ad: 'Kalem', fiyat: 12)
```

Başlatıcı listesi (3.5)

```
Kullanici(this.ad, String eposta)
    : eposta = eposta.trim().toLowerCase(),
      kayitYili = 2026 {
  print('Oluşturuldu');
}
```

Çalışma sırası:
1) parametreler ve `this.` → 2) başlatıcı listesi → 3) gövde

3.6 | final alanlar ve const kurucular

final alan

`final String siparisNo;`
Bir kez verilir, sonra değiştirilemez. Siparişin `durum`'u değişir, numarası değişmez.

03_08

`s.tutar = 100;` → derleme hatası

const kurucu

Tüm alanlar `final` ise:

```
const Konum(this.enlem, this.boylam);
const Konum magaza = Konum(37.93,
41.94);
```

Flutter bağlantısı

Widget'lar değişmez nesnelerdir: alanları `final`, kurucuları `const`. `const Text('Merhaba')` Flutter'a "bunu her seferinde yeniden oluşturma" der; VS Code'un sık sık "const ekleyin" önermesinin nedeni budur.

VS Code | 3.4–3.6 · Kurucu çeşitleri

Dosyalar

`03_05_kurucu_isimli_parametre.dart` · `03_06_baslatici_liste.dart` — **Birlikte yazalım**

`03_07_final_alanlar.dart` · `03_08_tuzak_final_alan.dart` — **Öğrenci denesin**

1. `Urun(fiyat: 45, ad: 'Defter', stok: 30)`: sıra neden önemsiz?
2. 03_06'da e-postayı boşluklu ve büyük harfli verin; temizlendiğini görün.
3. `s.tutar = 100;` satırının hata mesajını okuyun.

Sorgu

Başlatıcı listesinde `eposta = eposta.trim()`: soldaki ve sağdaki `eposta` aynı şey mi?

03_05

```
Süt (Süt Ürünleri): 32.50 TL, stok 12
Kalem (Genel): 12.00 TL, stok 0
Defter (Genel): 45.00 TL, stok 30
```

03_06

```
Yeni kullanıcı oluşturuldu: ayse.yilmaz
Ad           : Ayşe Yılmaz
E-posta      : ayse.yilmaz@ornek.com
Kullanıcı adı: ayse.yilmaz
Kayıt yılı   : 2026
```

03_07

```
Mağaza konumu: (37.93, 41.94)
SP-1024: 289.90 TL - Hazırlanıyor
SP-1024: 289.90 TL - Kargoda
```

03_08

```
'tutar' can't be used as a setter because it
    's final.
```

03_05

```
Süt (Süt Ürünleri): 32.50 TL, stok 12
Kalem (Genel): 12.00 TL, stok 0
Defter (Genel): 45.00 TL, stok 30
```

03_06

```
Yeni kullanıcı oluşturuldu: ayse.yilmaz
Ad           : Ayşe Yılmaz
E-posta      : ayse.yilmaz@ornek.com
Kullanıcı adı: ayse.yilmaz
Kayıt yılı   : 2026
```

03_07

```
Mağaza konumu: (37.93, 41.94)
SP-1024: 289.90 TL - Hazırlanıyor
SP-1024: 289.90 TL - Kargoda
```

03_08

```
'tutar' can't be used as a setter because it
    's final.
```

Cevap: Soldaki `eposta` **alan**, sağdaki **parametre**. Gövdedeki `print` çalıştığında tüm alanlar hazırdır.

3.7 | Kurucuların özeti

Yazım	Anlamı
(kurucu yazılmazsa)	Parametresiz varsayılan kurucu: `Urun()`
`Urun(this.ad, this.fiyat);`	Konumsal parametrelerle alanlara atama
`Urun({required this.ad, this.stok = 0});`	İsimli parametrelerle atama
`Urun(String ad) : ad = ad.trim();`	Başlatıcı listesi: değeri işleyerek atama
`Urun(...) { ... }`	Gövde: alanlar hazır olduktan sonraki işler
`const Konum(this.enlem, this.boylam);`	Tüm alanlar `final` ise sabit nesneler

4 | İsimli Kurucular ve factory

Nesneyi farklı yollarla ve kontrollü oluşturmak

Ders notu: Bölüm 4 · Dosyalar: `04_01` – `04_07`

4.1–4.2 | İsimli ve yönlendiren kurucular

```dart
class Konum {
  final double enlem, boylam;
  final String ad;

  Konum(this.enlem, this.boylam, this.ad);

  Konum.merkez()
      : enlem = 37.93, boylam = 41.94,
        ad = 'Siirt Merkez';
}

class Urun {
  Urun(this.ad, this.fiyat, this.stok);
  Urun.ucretsiz(String ad) : this(ad, 0, 999);
}
```

- Dart'ta **aynı adlı iki kurucu olmaz** (Java'daki gibi aşırı yükleme yok).
- İsimli kurucu, nesnenin **nasıl** oluşturulduğunu adıyla anlatır (4.1).
- Yönlendiren kurucu işi ana kurucuya devreder: `: this(...)` (4.2).

Flutter'da

```dart
EdgeInsets.all(16)
Image.network(adres)
ListView.builder(...)
```

4.3 | fromMap: Map'ten nesne oluşturmak

```dart
Urun.fromMap(Map<String, dynamic> veri)
    : ad = veri['ad'] as String,
      fiyat = (veri['fiyat'] as num).toDouble(),
      stok = veri['stok'] as int;

Map<String, dynamic> toMap() => {'ad': ad, 'fiyat': fiyat, 'stok': stok};
```

Neden as num?

Sunucudan fiyat 15 (`int`) gelebilir. `as double` çalışırken hata verir; `(as num).toDouble()` ikisini de kabul eder.

İki yön

`fromMap`: veri → nesne
`toMap`: nesne → veri (kaydetmek, göndermek)

VS Code | 4.1–4.3 · İsimli kurucular ve fromMap

Dosyalar

`04_01_isimli_kurucu.dart` · `04_02_yonlendiren_kurucu.dart` · `04_03_from_map.dart`
Birlikte yazalım

1. `Konum.merkez()` ve `Konum.koordinat(...)` ile nesne oluşturun.
2. `Urun.tukenmis(ad, fiyat)` yönlendiren kurucusunu yazın.
3. 04_03'te Ekmek'in fiyatını 15 bırakıp `as double` deneyin; hatayı görün, geri alın.

Dönüş | 4.1–4.3 · Çıktılar

04_01

```
Ev (37.925, 41.932)
Siirt Merkez (37.93, 41.94)
İsimsiz konum (37.95, 41.96)
```

04_02

```
Kulaklık: 749.90 TL (stok: 8)
Hediye Çanta: Ücretsiz (stok: 999)
Akıllı Saat: 2400.00 TL (stok: 0)
```

04_03

```
Süt - 32.50 TL (12 adet)
Ekmek - 15.00 TL (40 adet)
{ad: Süt, fiyat: 32.5, stok: 12}
```

Dönüş | 4.1–4.3 · Çıktılar

04_01

```
Ev (37.925, 41.932)
Siirt Merkez (37.93, 41.94)
İsimsiz konum (37.95, 41.96)
```

04_02

```
Kulaklık: 749.90 TL (stok: 8)
Hediye Çanta: Ücretsiz (stok: 999)
Akıllı Saat: 2400.00 TL (stok: 0)
```

04_03

```
Süt - 32.50 TL (12 adet)
Ekmek - 15.00 TL (40 adet)
{ad: Süt, fiyat: 32.5, stok: 12}
```

Tartışma

Asıl kurulum mantığı **tek yerde** (ana kurucuda) kaldı; isimli kurucular yalnızca kısayol. Ekmek'in fiyatı Map'te `15` (`int`) olarak geldi, ama nesnede `15.0` (`double`) olarak tutuluyor. Bunu hangi ifade sağladı?

4.4–4.5 | static üyeler ve factory kurucu

static (4.4)

Nesneye değil **sınıfa** aittir:

```
static int toplamSiparis = 0;
Siparis.toplamSiparis
```

static metotta `this` yoktur.

Normal kurucu	**factory kurucu**
Her çağrıda yeni nesne	Döndüreceği nesneye kendisi karar verir
`this.` ve başlatıcı listesi	`return` ile döndürür; `this` yok
Her zaman kendi sınıfı	Var olan nesneyi veya bir alt sınıfı döndürebilir

factory'nin dört kullanımı (4.5)

1) Değer temizleme ve doğrulama · **2)** Tekil nesne (singleton): `Ayarlar()` · **3)** Önbellek: `Renk.adiyla('mavi')` · **4)** Veriye göre alt sınıf seçmek (Bölüm 9)

4.5 | factory: doğrulama ve tekil nesne

Doğrulama (04_05)

```
Urun._(this.ad, this.fiyat);

factory Urun(String ad, double fiyat) {
  String temizAd = ad.trim();
  if (temizAd.isEmpty) temizAd = 'İsimsiz';
  double f = fiyat < 0 ? 0.0 : fiyat;
  return Urun._(temizAd, f);
}
```

Tekil nesne (04_06)

```
static final Ayarlar _tekOrnek =
    Ayarlar._ic();

factory Ayarlar() {
  return _tekOrnek;
}
```

`identical(a, b)`: aynı nesne mi?

VS Code | 4.4–4.5 · static ve factory

Dosyalar

`04_04_static.dart` · `04_05_factory_dogrulama.dart`	**Birlikte yazalım**
`04_06_factory_singleton.dart` · `04_07_factory_onbellek.dart`	**Öğrenci denesin**

1. Üçüncü bir sipariş ekleyin; numarası kaç olur?
2. `Urun('', 50)` ve `Urun('Kablo', -20)` çağrılarını deneyin.
3. `Ayarlar()` iki kez çağrılınca "Ayarlar yüklendi." kaç kez yazılır? Önce tahmin.
4. `Renk.adiyla('mavi')` ikinci kez çağrılınca ne olur?

Dönüş | 4.4–4.5 · Çıktılar

04_04

```
Sipariş 1: kargo 39.90 TL
Sipariş 2: kargo 0.00 TL
Toplam sipariş: 2
```

04_05

```
Kulaklık: 749.90 TL
İsimsiz ürün: 50.00 TL
Kablo: 0.00 TL
```

04_06

Dönüş | 4.4–4.5 · Çıktılar

04_04

```
Sipariş 1: kargo 39.90 TL
Sipariş 2: kargo 0.00 TL
Toplam sipariş: 2
```

04_05

```
Kulaklık: 749.90 TL
İsimsiz ürün: 50.00 TL
Kablo: 0.00 TL
```

04_06

```
Ayarlar yüklendi.
a2.tema: Koyu
Aynı nesne mi? true
```

04_07

```
Yeni renk oluşturuldu: mavi
Yeni renk oluşturuldu: yeşil
#1E88E5 #43A047
r1 ve r2 aynı mı? true
```

Dönüş | 4.4–4.5 · Çıktılar

04_04

```
Sipariş 1: kargo 39.90 TL
Sipariş 2: kargo 0.00 TL
Toplam sipariş: 2
```

04_05

```
Kulaklık: 749.90 TL
İsimsiz ürün: 50.00 TL
Kablo: 0.00 TL
```

04_06

```
Ayarlar yüklendi.
a2.tema: Koyu
Aynı nesne mi? true
```

04_07

```
Yeni renk oluşturuldu: mavi
Yeni renk oluşturuldu: yeşil
#1E88E5 #43A047
r1 ve r2 aynı mı? true
```

Tartışma

04_05'te private kurucu `Urun._` sayesinde dışarıdan **hatalı verili bir Urun oluşturmak imkânsız**. 04_06 ve 04_07'de factory **yeni nesne oluşturmadı**, var olanı döndürdü.

4.6 | Hangi kurucu ne zaman?

İhtiyaç	**Kurucu türü**	**Örnek**
Nesneyi alan değerleriyle oluşturmak	Normal	`Urun('Kalem', 12, 30)`
Aynı sınıfı farklı yollarla oluşturmak	İsimli	`Konum.merkez()`
Başka bir kurucuya kısayol	Yönlendiren	`Urun.ucretsiz('Canta')`
Map / JSON'dan nesne	İsimli veya factory	`Urun.fromMap(veri)`
Değişmeyecek sabit nesneler	`const`	`const Konum(37.93, 41.94)`
Doğrulama, singleton, önbellek, alt sınıf	`factory`	`Ayarlar()`

5 | Kapsülleme

Private alanlar, getter ve setter

Ders notu: Bölüm 5 · Dosyalar: `05_01` – `05_08`

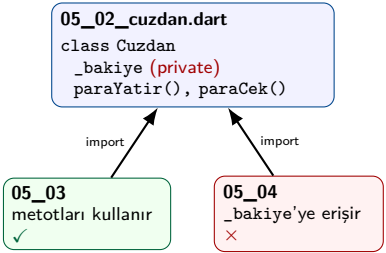
5.1–5.2 | Kapsülleme ve _ ile private

Sorun (5.1)

`c.bakiye = -5000;`
Kimse engellemiyor; `paraYatir`'daki kural tamamen atlanabiliyor.

Kapsülleme

Veriyi dışarıya kapatıp yalnızca **kontrollü yollarla** (metot, getter, setter) açmak.



Dart'ta private = dosya düzeyinde

_ ile başlayan ad **yalnızca aynı dosyadan** görülür (Java'daki gibi sınıf düzeyinde değil). Bu yüzden örnek **iki dosyadan** oluşuyor.

VS Code | 5.1–5.2 · İki dosyayla kapsülleme

Dosyalar

`05_01_kapsulleme_sorunu.dart`	**Önce tahmin**
`05_02_cuzdan.dart` *(kütüphane, main yok)* · `05_03_cuzdan_kullan.dart`	**Birlikte yazalım**
`05_04_tuzak_private.dart` · `05_05_ayni_dosya.dart`	**Hata mesajını oku**

1. `05_02` dosyasını oluşturun; **çalıştırmayın**, yalnızca kaydedin.
2. `05_03`'te `import '05_02_cuzdan.dart';` yazıp cüzdanı kullanın.
3. `05_04`'te `c._bakiye = -5000;` yazın: hata.
4. `05_05`'te aynı şeyi tek dosyada yapın: hata **yok**. Neden?

Dönüş | 5.1–5.2 · Çıktılar ve hata

05_01

```
Bakiye: -5000.0
```

05_03

```
true
false
true
false
Bakiye: 150.10 TL (2 işlem)
```

05_04 derleme hatası

```
The setter '_bakiye' isn't defined for the type '
    Cuzdan'.
```

05_05

```
0
```

Dönüş | 5.1–5.2 · Çıktılar ve hata

05_01

```
Bakiye: -5000.0
```

05_03

```
true
false
true
false
Bakiye: 150.10 TL (2 işlem)
```

05_04 derleme hatası

```
The setter '_bakiye' isn't defined for the type '
    Cuzdan'.
```

05_05

```
0
```

Tartışma

Başka dosyadan bakınca private alan **hiç yokmuş gibi** davranılır. Gerçek projelerde her sınıf kendi dosyasında durur.

5.3–5.4 | Getter ve setter

```
double _fiyat;
int _stok;

double get fiyat => _fiyat;          // okuma
double get kdvliFiyat => _fiyat * 1.2; // hesaplanan
bool get stoktaVar => _stok > 0;

set fiyat(double yeni) {             // yazma
  if (yeni < 0) return;              // kural
  _fiyat = yeni;
}
```

- **Getter** parantezsiz okunur: `urun.fiyat`
- **Hesaplanan getter**: alan değil, her okunuşta yeniden hesaplanır.
- **Setter** atama gibi kullanılır: `urun.fiyat = 2200;` Arkasında kural çalışır.
- Getter var, setter yoksa alan dışarıdan **salt okunur**.

Asıl fayda

Sınıfı kullanan kod hiç değişmeden, sınıfın içindeki kurallar değiştirilebilir.

VS Code | 5.3–5.5 · Getter ve setter

Dosyalar

`05_06_getter.dart` · `05_07_setter.dart` — **Birlikte yazalım**

`05_08_tuzak_getter_setter.dart` — **Hata mesajını oku**

1. Kulaklığı 3'er satın; `durum` getter'ının kendiliğinden değiştiğini görün.
2. `saat.fiyat = -100` ve `saat.indirimYuzdesi = 150` deneyin.
3. `c.bakiye()` ve `c.bakiye = 500` satırlarının hatalarını okuyun.

Sorgu

`durum` diye bir alan yok; değer nereden geliyor?

05_06

```
Kulaklık: 749.9 TL
KDV dahil: 899.88 TL
Durum: Stokta
Durum: Son 3 ürün!
Durum: Tükendi / stokta var mı? false
```

05_07

```
Uyarı: Fiyat negatif olamaz, değişiklik yapılmadı.
İndirim: %90
Akıllı Saat: 2200.00 TL → 1650.00 TL (%25 indirim)
```

05_06

```
Kulaklık: 749.9 TL
KDV dahil: 899.88 TL
Durum: Stokta
Durum: Son 3 ürün!
Durum: Tükendi / stokta var mı? false
```

05_07

```
Uyarı: Fiyat negatif olamaz, değişiklik yapılmadı.
İndirim: %90
Akıllı Saat: 2200.00 TL → 1650.00 TL (%25 indirim)
```

05_08: iki derleme hatası

`c.bakiye()` → getter **parantezsiz** çağrılır; `double` bir fonksiyon değildir.
`c.bakiye = 500` → `There isn't a setter named 'bakiye' in class 'Cuzdan'.` Bu bir hata değil, **tasarımın kendisi**: bakiye salt okunur.

5.6 | Getter mı, metot mu?

Getter	**Metot**
Nesnenin bir **özelliği** gibi okunuyorsa: `urun.kdvliFiyat`, `sepet.bosMu`	Bir **eylem** yapılıyorsa: `urun.sat(2)`
Hızlı ve yan etkisiz	Nesnenin durumunu değiştiriyorsa
Parametre gerekmiyor	Parametre gerekiyor

Flutter bağlantısı

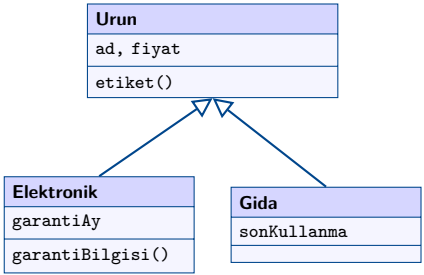
`TextEditingController.text` hem getter hem setter: okuyunca kutudaki yazıyı verir, `controller.text = ''` kutuyu temizler.

6 | Kalıtım

extends ve super

Ders notu: Bölüm 6 · Dosyalar: `06_01` – `06_05`

6.1–6.2 | Kod tekrarından kalıtıma



- `class Elektronik extends Urun`: Elektronik, Urun'un tüm alan ve metotlarını **miras alır** ve üstüne ekler.
- `etiket()` artık **tek yerde**.
- Üst sınıf (superclass) / alt sınıf (subclass).

"... bir ...'dur" testi

"Elektronik bir üründür" ✓
"Sepet bir üründür" $\times \rightarrow$ kalıtım değil

6.3 | super ile üst sınıfın kurucusu

```dart
class Elektronik extends Urun {
  int garantiAy;

  Elektronik(super.ad, super.fiyat, this.garantiAy);              // kısa yazım
  // Elektronik(String ad, double fiyat, this.garantiAy)
  //     : super(ad.trim(), fiyat);                               // değeri işleyerek
}
```

Çalışma sırası

Önce **üst sınıfın** kurucusu, sonra **alt sınıfın** gövdesi. Ev önce temelden kurulur.

06_04: super unutulursa

Dart üst sınıfın **parametresiz** kurucusunu arar; yoksa derleme hatası.

6.4: Her sınıfın atası `Object`; `toString`, `==`, `runtimeType` oradan gelir. Dart'ta **tek kalıtım** vardır.

VS Code | 6.1–6.4 · Kalıtım

Dosyalar

`06_01_kod_tekrari.dart` · `06_02_extends.dart` · `06_03_super_kurucu.dart` **Birlikte yazalım**

`06_04_tuzak_super.dart` · `06_05_object.dart` **Hata mesajını oku**

1. 06_01'deki iki sınıfı `Urun` üst sınıfıyla yeniden yazın (06_02).
2. 06_03'te iki kurucuya `print` ekleyin: hangisi önce çalışıyor?
3. 06_04'ü `super.ad, super.fiyat` ile düzeltin.

Sorgu

`etiket()` metodunun biçimini değiştirmek için 06_01'de kaç yer, 06_02'de kaç yer değişir?

Dönüş | 6.1–6.4 · Çıktılar ve hata

06_02

```
Kulaklık - 749.90 TL
24 ay garanti
Süt - 32.50 TL
Son kullanma: 15.10.2026
```

06_03

```
1) Urun kurucusu çalıştı: Tablet
2) Elektronik kurucusu çalıştı: 24 ay garanti
Oluşturulan: Tablet
```

06_04 derleme hatası

```
The superclass 'Urun' doesn't have a zero argument
     constructor.
```

06_05

```
Elektronik
Instance of 'Elektronik'
Kulaklık
```

Dönüş | 6.1–6.4 · Çıktılar ve hata

06_02

```
Kulaklık - 749.90 TL
24 ay garanti
Süt - 32.50 TL
Son kullanma: 15.10.2026
```

06_03

```
1) Urun kurucusu çalıştı: Tablet
2) Elektronik kurucusu çalıştı: 24 ay garanti
Oluşturulan: Tablet
```

06_04 derleme hatası

```
The superclass 'Urun' doesn't have a zero argument
     constructor.
```

06_05

```
Elektronik
Instance of 'Elektronik'
Kulaklık
```

Cevap: 06_01'de **iki**, 06_02'de **bir** yer. Ürün türü sayısı arttıkça fark büyür.

7 | Override

Metotları yeniden tanımlamak

Ders notu: Bölüm 7 · Dosyalar: `07_01` – `07_04`

7.1 | Override: aynı çağrı, farklı davranış

```
class Urun {
  double kargoUcreti() => 29.90;
}

class Elektronik extends Urun {
  @override
  double kargoUcreti() =>
      fiyat >= 1000 ? 0.0 : 49.90;
}

class Gida extends Urun {
  @override
  double kargoUcreti() => soguk ? 59.90 : 19.90;
}
```

- Alt sınıf, miras aldığı metodu **kendine göre** yeniden tanımlar.
- Hangi sürüm çalışır? Nesnenin **kendi sınıfındaki**.
- Override edilmeyen metotlar (`toString`) olduğu gibi miras kalır.

7.2–7.3 | @override neden önemli? Override kuralları

Yazım hatası (07_02)

```
@override
double kargoUcret() => 0;  // "i" eksik
```

Yeni bir metot oluştu; asıl metot override edilmedi. `@override` sayesinde VS Code **uyarır**. Yazılmasaydı hata sessizce kalırdı.

İmza uyuşmazlığı (07_03)

```
// Urun:        double kargoUcreti()
@override
int kargoUcreti() => 0;   // HATA
```

Programın geri kalanı `double` bekliyor; alt sınıf bu **sözü bozamaz**.

Kural: Override ettiğiniz **her** metodun üstüne `@override` yazın.

7.4 | super.metot() ile davranışı genişletmek

```
class Elektronik extends Urun {
  @override
  String etiket() => '${super.etiket()} | $garantiAy ay garanti';
}
```

İfade	Anlamı
`super(...)` / `super.ad` (kurucuda)	Üst sınıfın **kurucusunu** çağır
`super.etiket()` (metotta)	Üst sınıfın **metodunu** çağır
`this.ad`	**Bu** nesnenin alanı

Üst sınıfın etiket biçimi değişirse alt sınıfların etiketleri de **kendiliğinden** güncellenir.

VS Code | 7.1–7.4 · Override

Dosyalar

`07_01_override.dart` · `07_04_super_metot.dart` — **Birlikte yazalım**

`07_02_tuzak_override_yazim.dart` · `07_03_tuzak_override_imza.dart` — **Uyarıyı / hatayı oku**

1. Beş ürün için kargo ücretlerini önce tahmin edin, sonra çalıştırın.
2. 07_02'de `@override` satırını silin: uyarı kayboluyor mu? Hata düzeldi mi?
3. `IndirimliUrun` sınıfını `super.etiket()` kullanarak yazın.

Dönüş | 7.1–7.4 · Çıktılar

07_01

```
Kitap (120.00 TL) → kargo: 29.90 TL
Tablet (8999.00 TL) → kargo: 0.00 TL
Kablo (129.90 TL) → kargo: 49.90 TL
Dondurma (85.00 TL) → kargo: 59.90 TL
Makarna (25.00 TL) → kargo: 19.90 TL
```

07_02 uyarı ve çıktı

```
The method doesn't override an inherited method.
29.9
```

07_04

```
Defter - 45.00 TL
Kulaklık - 749.90 TL | 24 ay garanti
Mont - 1800.00 TL → 1260.00 TL (%30)
```

Dönüş | 7.1–7.4 · Çıktılar

07_01

```
Kitap (120.00 TL) → kargo: 29.90 TL
Tablet (8999.00 TL) → kargo: 0.00 TL
Kablo (129.90 TL) → kargo: 49.90 TL
Dondurma (85.00 TL) → kargo: 59.90 TL
Makarna (25.00 TL) → kargo: 19.90 TL
```

07_02 uyarı ve çıktı

```
The method doesn't override an inherited method.
29.9
```

07_04

```
Defter - 45.00 TL
Kulaklık - 749.90 TL | 24 ay garanti
Mont - 1800.00 TL → 1260.00 TL (%30)
```

07_02: `@override` silinince uyarı kaybolur ama **hata devam eder**: tablet hâlâ 29.9 TL kargo öder.

7.4 | Flutter bağlantısı: her ekran bir override'dır

```dart
class KarsilamaEkrani extends StatelessWidget {
  const KarsilamaEkrani({super.key});

  @override
  Widget build(BuildContext context) {
    return const Text('Merhaba');
  }
}
```

`extends`	Kalıtım (6)
`const`	const kurucu (3.6)
`super.key`	Üst sınıfa iletme (6.3)
`@override`	Override (7)
`build`	Flutter'ın çağırdığı metot

Bu haftanın konuları Flutter'ın temelidir

Flutter, ekranı çizmesi gerektiğinde **sizin** `build` metodunuzu çağırır.

8 | Polimorfizm ve is

Aynı çağrı, nesnenin gerçek tipine göre farklı davranış

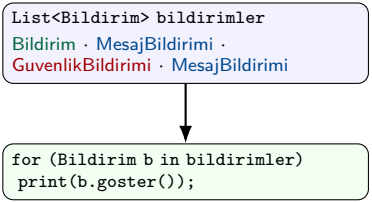
Ders notu: Bölüm 8 · Dosyalar: `08_01` – `08_09`

8.1 | Statik tip ve gerçek tip

```
Urun u = Elektronik('Kulaklık', 24);   // her Elektronik bir Urun'dur
print(u.tanit());                       // Elektronik'teki sürüm çalışır
print(u.garantiAy);                     // HATA: Urun'da garantiAy yok
```

Soru	**Statik tip (`Urun`)**	**Gerçek tip (`Elektronik`)**
Hangi üyelere **erişebilirim**?	✓	
Hangi metot sürümü **çalışır**?		✓

8.2 | Polimorfizm: tek liste, farklı davranışlar



- Döngü, hangi bildirimin hangi türde olduğunu **bilmiyor**.
- Her nesneye "kendini göster" der; her nesne **kendi bildiği** gibi gösterir.
- Yeni tür eklemek için yalnızca yeni sınıf; döngüye **dokunulmaz**.

Flutter bağlantısı

`Column(children: [Text(...), Icon(...), ElevatedButton(...)])`: `children` bir `List<Widget>`. Flutter her birine "kendini çiz" der.

VS Code | 8.1–8.2 · Üst tip ve polimorfizm

Dosyalar

`08_01_ust_tip.dart` · `08_02_tuzak_ust_tip.dart` · `08_03_polimorfizm.dart` **Birlikte yazalım**

1. `Urun u = Elektronik(...)`: `u.tanit()` ve `u.runtimeType` ne yazar?
2. `u.garantiAy` satırının hatasını okuyun.
3. 08_03'e `KampanyaBildirimi` ekleyin (öncelik 1, `[Kampanya]` ön eki). Döngüyü değiştirmeniz gerekti mi?

Dönüş | 8.1–8.2 · Çıktılar

08_01

```
Kulaklık bir elektronik üründür (24 ay garanti)
    .
Elektronik
```

08_02 derleme hatası

```
The getter 'garantiAy' isn't defined for the
    type 'Urun'.
```

08_03

```
[Bildirim] Uygulama güncellendi
[Mesaj] Ayşe: Akşam toplantı var mı?
[GÜVENLİK] Yeni cihazdan giriş yapıldı
[Mesaj] Mert: Ödev dosyası ekte
Toplam öncelik puanı: 8
```

08_01

```
Kulaklık bir elektronik üründür (24 ay garanti)
      .
Elektronik
```

08_02 derleme hatası

```
The getter 'garantiAy' isn't defined for the
     type 'Urun'.
```

08_03

```
[Bildirim] Uygulama güncellendi
[Mesaj] Ayşe: Akşam toplantı var mı?
[GÜVENLİK] Yeni cihazdan giriş yapıldı
[Mesaj] Mert: Ödev dosyası ekte
Toplam öncelik puanı: 8
```

Cevap

`KampanyaBildirimi` için yalnızca yeni bir sınıf yazıldı; döngüye **tek satır bile** dokunulmadı. Polimorfizmin asıl değeri budur.

8.3–8.5 | is, is!, as ve tip desenleri

```dart
for (Urun u in sepet) {
  if (u is Elektronik) {
    print(u.garantiAy);   // u yükseltildi
  } else if (u is Gida) {
    print(u.sonKullanma);
  }
}

String ekBilgi(Urun u) => switch (u) {
      Elektronik e => '${e.garantiAy} ay',
      Gida g => 'SKT: ${g.sonKullanma}',
      _ => 'Ek bilgi yok',
    };
```

`is`	Bu tipte mi? Doğruysa **tip yükseltme**
`is!`	Bu tipte değil mi?
`as`	Bu tipe dönüştür; değilse **çalışma zamanı hatası**

08_06

`Gida` nesnesine `as Elektronik` → program çöker. Emin değilseniz `is`.

8.6 | Tuzak: nesnenin alanları yükseltilemez

Derleme hatası (08_08)

```
if (kalem.urun is Elektronik) {
  print(kalem.urun.garantiAy);
}
```

Doğrusu (08_09)

```
Urun u = kalem.urun;
if (u is Elektronik) {
  print(u.garantiAy);
}
```

Neden? `kalem.urun` yerel değişken değil, bir nesnenin **alanıdır**. Dart'ın gözünde kontrol ile kullanım arasında değişebilir. Çözüm: Değeri önce **yerel bir değişkene** alın.

VS Code | 8.3–8.6 · Tip kontrolü

Dosyalar

`08_04_is_tip_kontrolu.dart` · `08_05_as_is_degil.dart` · `08_06_tuzak_as.dart`
`08_07_switch_tip_deseni.dart` · `08_08_tuzak_alan_yukseltme.dart` ·
`08_09_alan_yukseltme_dogru.dart`

1. 08_04'te `if` bloğu dışında `u.garantiAy` yazmayı deneyin.
2. 08_06'yı çalıştırın: hata derleme sırasında mı, çalışırken mi?
3. 08_04'teki `if-else if` zincirini 08_07'deki switch ile karşılaştırın.
4. 08_08'i yerel değişkenle düzeltin.

Dönüş | 8.3–8.6 · Çıktılar ve hatalar

08_04

```
Kulaklık: 24 ay garanti
Süt: SKT 15.10.2026
Defter: ek bilgi yok
Yoğurt: SKT 12.10.2026
Sepette 2 gıda ürünü var.
```

08_05

```
Tablet bir gıda ürünü değil.
Garanti: 24 ay
```

08_07

```
Kulaklık → 24 ay garanti
Süt      → SKT: 15.10.2026
Defter   → Ek bilgi yok
```

08_06 çalışma zamanı hatası

```
Unhandled exception:
type 'Gida' is not a subtype of type 'Elektronik' in
     type cast
```

08_08 derleme hatası

```
The getter 'garantiAy' isn't defined for the type '
     Urun'.
```

08_09

```
Kulaklık: 24 ay garanti
```

8.7 | is mi, override mı?

`is` zinciriyle	Override ile (polimorfizm)
`if (u is Elektronik) {...} else if (u is Gida) {...}`	`kargo = u.kargoUcreti();`
Yeni tür eklenince bu zincirin olduğu **her yer** değişir	Yeni tür kendi metodunu yazar; mevcut koda dokunulmaz
Unutulan yer sessizce yanlış sonuç verir	Unutmak mümkün değil: kural sınıfın içinde

Kural

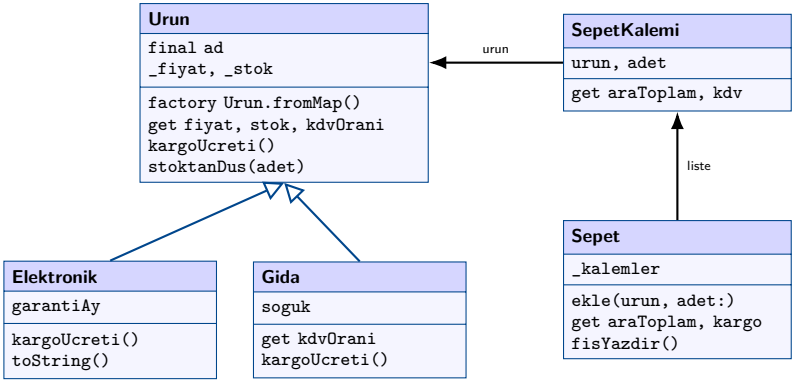
Davranış türe göre değişiyorsa → **metot + override**.
Yalnızca alt türe özgü bir **veriye** (garanti süresi) erişmek gerekiyorsa → `is`.

9 | Hepsini Birleştirelim

Cep Market v2: Hafta 3'ün sepeti, nesnelerle

Ders notu: Bölüm 9 · Dosyalar: 09_01

9 | Sınıf diyagramı



Üçgen ok: kalıtım ("... bir ...'dur"). Düz ok: bir nesne başka bir nesneyi **tutuyor** ("... sahiptir").

Dosya

`09_01_cep_market_oop.dart` **Öğretim elemanı + sınıf**

1. **Adım 1:** `Urun`: private fiyat/stok, getter'lar, `kargoUcreti()`, `stoktanDus()`.
2. **Adım 2:** `Elektronik` ve `Gida`: `super` parametreleri, override'lar.
3. **Adım 3:** `factory Urun.fromMap`: `'tur'` değerine göre **alt sınıf seçimi**.
4. **Adım 4:** `SepetKalemi` ve `Sepet`: hesaplanan getter'lar, `fisYazdir()`.
5. **Adım 5:** `main`: katalog verisi, sepete ekleme, fiş.

Sorgu

`Sepet` sınıfında `is Elektronik` ya da `is Gida` diye soran bir satır var mı? Neden yok?

Dönüş | 9 · Çıktı

```
Katalog:
- Kulaklık (24 ay garanti): 749.90 TL (Elektronik)
- Süt: 32.50 TL (Gida)
- Dondurma (soğuk zincir): 85.00 TL (Gida)
- Defter: 45.00 TL (Urun)

Kulaklık x1 sepete eklendi.
Süt x2 sepete eklendi.
Dondurma: Yetersiz stok (2 adet var).
Dondurma x2 sepete eklendi.
Defter x2 sepete eklendi.
```

```
=============== FİŞ ===============
Kulaklık (24 ay garanti) x1    749.90
Süt                      x2     65.00
Dondurma (soğuk zincir)  x2    170.00
Defter                   x2     90.00
-----------------------------------
Ara toplam : 1074.90 TL
KDV        : 170.33 TL
Kargo      : 59.90 TL
TOPLAM     : 1305.13 TL
Dondurma stoğu: 0
```

Dönüş | 9 · Çıktı

```
Katalog:
- Kulaklık (24 ay garanti): 749.90 TL (Elektronik)
- Süt: 32.50 TL (Gida)
- Dondurma (soğuk zincir): 85.00 TL (Gida)
- Defter: 45.00 TL (Urun)

Kulaklık x1 sepete eklendi.
Süt x2 sepete eklendi.
Dondurma: Yetersiz stok (2 adet var).
Dondurma x2 sepete eklendi.
Defter x2 sepete eklendi.
```

```
=============== FİŞ ===============
Kulaklık (24 ay garanti) x1    749.90
Süt                      x2     65.00
Dondurma (soğuk zincir)  x2    170.00
Defter                   x2     90.00
-----------------------------------
Ara toplam : 1074.90 TL
KDV        : 170.33 TL
Kargo      : 59.90 TL
TOPLAM     : 1305.13 TL
Dondurma stoğu: 0
```

Üç güç bir arada

Factory: `fromMap` veriye bakıp doğru alt sınıfı seçti (`runtimeType` sütunu). **Polimorfizm:** `Sepet` hiç tip sormadı; KDV gıdada %1, diğerlerinde %20; kargo en yüksek olan (soğuk zincir 59.90). **Kapsülleme:** 5 dondurma reddedildi, stok değişmedi.

9 | Deneyin: Kitap sınıfı eklemek

Görev

`Kitap extends Urun`:
KDV oranı %0, kargo 14.90 TL.
`fromMap`'e `'kitap'` kolunu ekleyin.

Gözlem

`Sepet` sınıfında **hiçbir değişiklik** gerekmedi.

Flutter bağlantısı

`Urun`, `SepetKalemi`, `Sepet` bir Flutter alışveriş uygulamasının **model katmanına** neredeyse olduğu gibi taşınır. Ekranlar bu nesneleri gösterir; **kurallar** ekranlarda değil, sınıfların içinde yaşar.

10 | Örnek Problem

Ödeme yöntemleri: aynı çıktıya üç tasarım

Ders notu: Bölüm 10 · Dosyalar: `10_01` – `10_03`

10 | Problem: ödeme yöntemleri ve işlem ücretleri

Yöntem	Ekranda görünen ad	İşlem ücreti
Kredi kartı	`Kredi Kartı (tek çekim)` / `(3 taksit)`	%2; taksit $>$ 1 ise ayrıca %3 vade farkı
Havale	`Havale/EFT`	Sabit 5 TL
Kapıda ödeme	`Kapıda Ödeme`	Sabit 25 TL
Cep Cüzdan	`Cep Cüzdan`	Ücretsiz

Beklenen ekran çıktısı

```
Kredi Kartı (3 taksit) : 1200.00 TL + 60.00 TL = 1260.00 TL
Kredi Kartı (tek çekim): 450.00 TL + 9.00 TL = 459.00 TL
Havale/EFT             : 800.00 TL + 5.00 TL = 805.00 TL
Kapıda Ödeme           : 320.00 TL + 25.00 TL = 345.00 TL
Cep Cüzdan             : 150.00 TL + 0.00 TL = 150.00 TL
Toplam işlem ücreti: 99.00 TL
```

10 | Sıra sizde: çözümden önce 15 dakika

1. Ödemeleri nasıl temsil edeceğinize karar verin: tek sınıf mı, alt sınıflar mı?
2. Ücret kuralı nerede duracak: sınıfın içinde mi, dışarıdaki bir fonksiyonda mı?
3. `padRight(23)` ile hizalayıp beş ödemeyi ve toplam ücreti yazdırın.

Düşünmeniz için

Yarın "Kripto" diye yeni bir ödeme yöntemi eklenecek. Sizin tasarımınızda kaç yeri değiştirmeniz gerekir?

VS Code | 10 · Üç çözüm

Dosyalar

`10_01_odeme_tek_sinif.dart`	**Tür alanı + switch**
`10_02_odeme_polimorfizm.dart`	**Kalıtım + override**
`10_03_odeme_tip_deseni.dart`	**Kalıtım + dışarıda tip desenleri**

1. Üç çözümün `main` fonksiyonlarını yan yana açın: ne kadar benziyorlar?
2. 10_01'de `'havale'` yerine `'havle'` yazın. Ne oldu?
3. 10_02'de `Havle(800)` yazın. Ne oldu?

Dönüş | 10 · Üç tasarımın karşılaştırması

Ölçüt	**1. Tek sınıf**	**2. Polimorfizm**	**3. Tip desenleri**
Kod uzunluğu	En kısa	En uzun	Orta
"Kripto" eklemek	2 switch değişir	**Yalnızca yeni sınıf**	Yeni sınıf + 2 fonksiyon
Tür adında yazım hatası	Sessizce ücretsiz	Derleme hatası	Derleme hatası
Kurallar nerede?	Tek sınıfta, karışık	**Kendi sınıfında**	Dışarıdaki fonksiyonlarda

Dönüş | 10 · Üç tasarımın karşılaştırması

Ölçüt	1. Tek sınıf	2. Polimorfizm	3. Tip desenleri
Kod uzunluğu	En kısa	En uzun	Orta
"Kripto" eklemek	2 switch değişir	**Yalnızca yeni sınıf**	Yeni sınıf + 2 fonksiyon
Tür adında yazım hatası	Sessizce ücretsiz	Derleme hatası	Derleme hatası
Kurallar nerede?	Tek sınıfta, karışık	**Kendi sınıfında**	Dışarıdaki fonksiyonlarda

Sonuç

Üç çözüm de aynı çıktıyı üretir. Sürekli yenisi eklenen türler için **2. çözüm** en sağlamıdır. 3. çözümün zayıf noktası (unutulan tür _ koluna düşer), gelecek hafta göreceğimiz `sealed` sınıflarla derleme hatasına dönüşür.

11–14 | Pekiştirme ve Ödev

Özet, kendinizi test edin, Ödev 3, kaynaklar

Ders notu: Bölüm 11–14 · Dosyalar: –

11 | Hızlı özet

Konu	Sözdizimi	Not
Sınıf, nesne	`class Urun {...}` · `Urun(...)`	`new` yazılmaz
Kurucu	`Urun(this.ad, this.fiyat);`	Gövdede atama olmaz
İsimli / factory	`Konum.merkez()` · `factory Urun(...)`	factory `return` eder
Private	`double _bakiye;`	Dosya düzeyinde
Getter / setter	`get fiyat => _fiyat;` · `set fiyat(v)`	Parantezsiz / atama gibi
Kalıtım	`extends` · `super.ad` · `super(...)`	Tek kalıtım
Override	`@override` · `super.etiket()`	Her zaman `@override`
Polimorfizm	`List<Urun>` içinde alt sınıflar	Gerçek tip çalışır
Tip kontrolü	`is` · `is!` · `as` · `Gida g =>`	Alanlar yükseltilmez

12 | Kendinizi test edin: Sınıf ve kurucular

S2 `Kitap(String baslik) { this.baslik = baslik; }` neden derlenmez?

S4 `ad = ad;` satırı neden alanı değiştirmez?

S5 Normal kurucu ile `factory` kurucu arasındaki iki fark nedir?

S6 `_bakiye` alanını hangi kod görebilir?

12 | Kendinizi test edin: Sınıf ve kurucular

S2 `Kitap(String baslik) { this.baslik = baslik; }` neden derlenmez?

S4 `ad = ad;` satırı neden alanı değiştirmez?

S5 Normal kurucu ile `factory` kurucu arasındaki iki fark nedir?

S6 `_bakiye` alanını hangi kod görebilir?

Kısa cevaplar

S2 Alan gövdeden önce hazır olmalı; `Kitap(this.baslik);`. **S4** İki `ad` de parametre; `this.ad = ad`. **S5** factory her seferinde yeni nesne oluşturmak zorunda değil ve `this` kullanamaz. **S6** Yalnızca **aynı dosyadaki** kod.

Ayrıntılı cevaplar: **Ders notu Bölüm 12**.

12 | Kendinizi test edin: Kalıtım ve polimorfizm

S8 `class B extends A { B(this.y); }` neden derlenmeyebilir?

S9 `@override` yazmamak programı bozar mı? Neden yine de yazılmalı?

S10 `Hayvan`, `Kedi`, `Yavru` listesinde `h.ses()` ne yazar?

S12 `is` ile `as` arasındaki fark nedir?

12 | Kendinizi test edin: Kalıtım ve polimorfizm

S8 `class B extends A { B(this.y); }` neden derlenmeyebilir?

S9 `@override` yazmamak programı bozar mı? Neden yine de yazılmalı?

S10 `Hayvan`, `Kedi`, `Yavru` listesinde `h.ses()` ne yazar?

S12 `is` ile `as` arasındaki fark nedir?

Kısa cevaplar

S8 `A`'nın parametresiz kurucusu yoksa `super` çağrısı gerekir. **S9** Bozmaz, ama yazım hatası sessizce kalır. **S10** `...`, `Miyav`, `Miyav miyav`. **S12** `is` soru sorar ve tipi yükseltir; `as` iddiadır, yanlışsa program çöker.

13 | Ödev 3: Teslim ve kapsam

AI kuralı

AI kavram sormak ve hata açıklatmak için kullanılabilir; **kod yazdırmak yasaktır**. Çözümler 5. haftanın ilk 15 dakikasında.

- Her soru ayrı `odev3_s1.dart`, `odev3_s2.dart`, ... dosyasında.
- Verilen `main` fonksiyonunu **aynen** kullanın; çıktı birebir aynı olmalı.
- Override ettiğiniz her metotta `@override`; Problems panelinde hata ve uyarı kalmasın.
- Son tarih ve teslim yöntemi Classroom'daki ödev sayfasında.

13 | Ödev 3: Sorular

Soru	Kapsam
1 Akıllı ampul	Private alan, getter, setter ile 0–100 aralığı, durum getter'ı
2 Kullanıcı profilleri	İsimli kurucular, `factory Kullanici.epostadan`, `fromMap`, `toString`
3 Ulaşım uygulaması	`Taksi`, `Scooter`, `Otobus`: kalıtım, override, `super.ozet()`
4 Medya kitaplığı	`Sarki`, `Podcast`, `Video`; `is` ile tür sayma
5 Spor salonu	factory ile alt sınıf seçimi; `OgrenciUyelik extends Aylik`; `super.aylikUcret()`

Her sorunun `main` fonksiyonu ve beklenen çıktısı ders notunun **13. bölümünde**.

14 | Kaynaklar ve gelecek hafta

Dart resmi belgeleri

- `https://dart.dev/language/classes`
- `https://dart.dev/language/constructors`
- `https://dart.dev/language/methods`
- `https://dart.dev/language/extend`
- `https://dart.dev/language/pattern-types`

Hafta 5 | OOP II ve Fonksiyonel Programlama

`abstract`, `implements`, `mixin`, `extension`, `enum`, `sealed`; lambda, `where`, `map`, `fold`, `sort`. Derse gelmeden **Ödev 3**'ü ve ders notunun **12. bölümünü** tamamlayın.

Önce tahmin et.
Sonra çalıştır.
Nedenini açıkla.

Sonraki adım: **Ödev 3** + **Hafta 5** hazırlığı